tengo la siguiente tarea:

Patrones de Diseño:

1. Elegir un patrón
2. Mostrar el patrón genérico
3. Implementar el patrón para una situación específica
4. Mostrar patrón aplicado en la situación implementada

Todo esto tiene que estar en diapositivas powerpoint

Elegimos el patrón Strategy

Mostrar más

Primero dame el contenido y revisemos todo, por ej: tenemos que implementarlo en una situación específica tipo un ejemplo real, dame ideas y escogeré uno. Organizamos todo y una vez que esté bien definido me pasas las diapos (sencillas, sin elementos exagerados, con buen texto pensado que no me pasarás guión porque lo mostraré de una en clase sin estudiarlo), pero sin explotar de texto, las distribuiras bien

Perfecto, Strategy es una buena elección: es claro, tiene muchos ejemplos reales y se entiende bien en diapositivas. Vamos por pasos, como pediste.

1. Contenido base del patrón (para que lo revisemos)

Intención: Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. Permite cambiar el comportamiento en tiempo de ejecución sin modificar la clase que lo usa.

Problema que resuelve: Cuando una clase tiene varias formas de hacer lo mismo y se llena de `if/else` o `switch`. Cada vez que aparece una variante nueva hay que modificar la clase (viola el principio Abierto/Cerrado).

Estructura genérica (UML):

- **Context:** mantiene una referencia a una Strategy y delega el trabajo en ella.
- **Strategy (interfaz):** declara el método común (`execute()`).
- **ConcreteStrategyA / B / C:** implementan cada algoritmo.
- **Client:** elige qué estrategia pasarle al Context.

Código genérico (pseudocódigo/Java):

```java
interface Strategy { void execute(); }

class ConcreteStrategyA implements Strategy {
    public void execute() { /* algoritmo A */ }
}
class ConcreteStrategyB implements Strategy {
    public void execute() { /* algoritmo B */ }
}

class Context {
    private Strategy strategy;
    public void setStrategy(Strategy s) { this.strategy = s; }
    public void doWork() { strategy.execute(); }
}
```

Ventajas: elimina condicionales, cumple Abierto/Cerrado, algoritmos intercambiables en ejecución, fácil de probar por separado.
Desventajas: más clases, el cliente debe conocer las estrategias, puede ser excesivo si solo hay 2 variantes que no cambian.

---

2. Ideas de situaciones reales (elige una)

1. **Métodos de pago en una tienda online:** tarjeta, PayPal, transferencia, criptomonedas. Cada uno procesa el pago distinto. *(El clásico, muy fácil de entender para la clase.)*
2. **Cálculo de costo de envío:** estándar, express, internacional, retiro en tienda. Cambia el algoritmo de precio según la opción.
3. **Descuentos en un carrito de compras:** sin descuento, por cupón, por Black Friday, cliente VIP.
4. **Algoritmos de ordenamiento:** burbuja, quicksort, merge sort, elegidos según el tamaño de datos. *(Muy académico, menos "real".)*
5. **Navegación/rutas tipo Google Maps:** a pie, en auto, en bus, en bicicleta; cada modo calcula la ruta distinto.
6. **Compresión de archivos:** ZIP, RAR, 7z, TAR.
7. **Validación de contraseñas / autenticación:** usuario-clave, Google, huella, token.
8. **Videojuego:** un enemigo con comportamiento agresivo, defensivo o sigiloso que cambia en pleno juego.

Mi recomendación: la **1 (métodos de pago)** o la **5 (rutas de Google Maps)**. La 1 es la más fácil de explicar y codificar; la 5 es más llamativa y la gente la reconoce de inmediato. Si quieren algo que se vea más "vivo" en clase, la 8 también funciona bien.

---

3. Propuesta de estructura de diapositivas

Siguiendo exactamente los 4 puntos de la tarea, unas 11 diapositivas:

#	Diapositiva	Contenido
1	Portada	Título, integrantes, curso, fecha
2	Patrón elegido	Strategy: categoría (comportamiento) y una frase de intención
3	¿Qué problema resuelve?	Idea del `if/else` gigante, con un mini ejemplo
4	Estructura genérica	Diagrama UML con Context, Strategy y ConcreteStrategies
5	Código genérico	Interfaz + 2 estrategias + Context
6	Ventajas y desventajas	Dos columnas cortas
7	Situación específica	Descripción del escenario real elegido
8	Problema sin patrón	Código con `if/else` (el "antes")
9	Implementación	Código con Strategy (el "después")
10	Patrón aplicado	UML con los nombres reales del ejemplo (equivalencia: Context = X, Strategy = Y)
11	Conclusiones	3 ideas clave

Opcional: una diapositiva de ejecución/demo con la salida del programa.

Para avanzar necesito que me confirmes:

1. **¿Qué situación eliges** (o quieres otra idea)?
2. **¿Qué lenguaje** quieren para el código (Java, Python, C#, JavaScript...)?
3. **¿Te parece bien la estructura** de 11 diapositivas, o quitas/agregas alguna?

Con eso armo el código completo y el texto de cada diapositiva para que lo revisemos antes de generar el PowerPoint.

elejire el 1 y el 8 (ahora tengo que explicar 2 problemas reales de ejemplo porque somos mas integrantes). añade eso y ten en cuenta que usaremos un lenguaje de programacion para explicarlo. en que lenguaje es mas facil de explicar en sintaxis

Comparando Python y Java para el patrón Strategy

Lenguaje: mi recomendación es Python

Para explicar en clase, Python es el más fácil de leer porque casi no tiene "ruido" sintáctico: no hay `public`, `void`, `;` ni llaves, y el código se lee casi como pseudocódigo. Compara la misma estrategia:

Python

```python
class Tarjeta(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando {monto} con tarjeta")
```

Java

```java
public class Tarjeta implements MetodoPago {
    public void pagar(double monto) {
        System.out.println("Pagando " + monto + " con tarjeta");
    }
}
```

	Python	Java
Legibilidad en diapositiva	Muy alta, cabe más código por slide	Media, más verboso
Se nota el patrón (interfaz, `implements`)	Se puede mostrar con `ABC`	Muy explícito, se parece al UML
Mejor si...	Quieren que se entienda rápido	Su curso es de POO en Java y el profesor lo espera

Mi consejo: Python con una clase abstracta (`ABC`), así se ve claramente la "interfaz" sin tanto código. Pero si su curso usa Java, usen Java para ser coherentes con la materia.

Estructura actualizada (15 diapositivas)

La parte genérica se explica una sola vez, y luego cada ejemplo repite el mismo esquema de 4 pasos. Así cada integrante o subgrupo puede encargarse de un ejemplo.

#	Diapositiva	Contenido
1	Portada	Título, integrantes, curso, fecha
2	Patrón elegido	Strategy, categoría (comportamiento) e intención en una frase
3	¿Qué problema resuelve?	El `if/else` que crece con cada variante
4	Estructura genérica	UML: Context, Strategy, ConcreteStrategies
5	Código genérico	Interfaz + 2 estrategias + Context
6	Ventajas y desventajas	Dos columnas cortas
Ejemplo 1: Métodos de pago		
7	Situación	Tienda online: tarjeta, PayPal, transferencia
8	Sin patrón (antes)	Clase `Pedido` con `if/else` por método de pago
9	Con Strategy (después)	Interfaz `MetodoPago` + clases concretas + `Pedido`
10	Patrón aplicado	UML con nombres reales y equivalencia (Context = `Pedido`, Strategy = `MetodoPago`...)
Ejemplo 2: Enemigo de videojuego		
11	Situación	Enemigo que cambia de comportamiento en pleno juego
12	Sin patrón (antes)	Clase `Enemigo` con `if/else` por comportamiento
13	Con Strategy (después)	`Comportamiento` + Agresivo, Defensivo, Sigiloso + `Enemigo`
14	Patrón aplicado	UML con nombres reales; destacar el cambio en tiempo de ejecución (ej. con poca vida pasa de agresivo a defensivo)
15	Conclusiones	3 ideas clave

Un detalle útil: los dos ejemplos muestran cosas distintas del patrón. En pagos, el cliente **elige** la estrategia una vez. En el enemigo, la estrategia **cambia dinámicamente** durante la ejecución. Eso hace que los ejemplos se complementen en lugar de repetirse.

Opcional: agregar una diapositiva de demo con la salida del programa al final de cada ejemplo (serían 17).

Para seguir, confírmame:

1. **¿Python o Java?** (o el lenguaje que use su curso)
2. **¿Te parece bien la estructura de 15 diapositivas**, con o sin las dos de demo?

Con eso te paso el código completo de ambos ejemplos (antes y después) y el texto de cada diapositiva para revisarlo antes de generar el PowerPoint.

lo haremos en python.

Escribiendo el código de los ejemplos en diapositivas

Código de los dos ejemplos (Python)

Ejemplo 1: Métodos de pago

Antes (sin patrón)

```python
class Pedido:
    def __init__(self, monto):
        self.monto = monto

    def pagar(self, metodo):
        if metodo == "tarjeta":
            print(f"Pagando ${self.monto} con tarjeta")
        elif metodo == "paypal":
            print(f"Pagando ${self.monto} con PayPal")
        elif metodo == "transferencia":
            print(f"Pagando ${self.monto} por transferencia")
        else:
            raise ValueError("Método no soportado")
```

Después (con Strategy)

```python
```

```python
from abc import ABC, abstractmethod

class MetodoPago(ABC):
    @abstractmethod
    def pagar(self, monto): ...

class Tarjeta(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} con tarjeta")

class PayPal(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} con PayPal")

class Transferencia(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} por transferencia")

class Pedido:
    def __init__(self, monto, metodo):
        self.monto = monto
        self.metodo = metodo

    def pagar(self):
        self.metodo.pagar(self.monto)

# Uso
pedido = Pedido(100, PayPal())
pedido.pagar()   # Pagando $100 con PayPal
```

Ejemplo 2: Enemigo de videojuego

Antes (sin patrón)

python

```python
class Enemigo:
    def __init__(self):
        self.vida = 100
        self.modo = "agresivo"

    def actuar(self):
        if self.modo == "agresivo":
            print("Ataca al jugador")
        elif self.modo == "defensivo":
            print("Se cubre y retrocede")
        elif self.modo == "sigiloso":
            print("Se oculta y acecha")
```

Después (con Strategy)

python

```python
from abc import ABC, abstractmethod

class Comportamiento(ABC):
    @abstractmethod
    def actuar(self): ...

class Agresivo(Comportamiento):
    def actuar(self):
        print("Ataca al jugador")

class Defensivo(Comportamiento):
    def actuar(self):
        print("Se cubre y retrocede")

class Sigiloso(Comportamiento):
    def actuar(self):
        print("Se oculta y acecha")

class Enemigo:
    def __init__(self, comportamiento):
        self.vida = 100
        self.comportamiento = comportamiento

    def actuar(self):
        self.comportamiento.actuar()

    def recibir_dano(self, dano):
        self.vida -= dano
        if self.vida < 30:
            self.comportamiento = Defensivo()   # cambio en ejecución

# Uso
enemigo = Enemigo(Agresivo())
enemigo.actuar()          # Ataca al jugador
enemigo.recibir_dano(80)
enemigo.actuar()          # Se cubre y retrocede
```

Una nota honesta: en el enemigo, la decisión de *cuándo* cambiar sigue siendo un `if` dentro de `recibir_dano`. Strategy no elimina esa decisión, elimina el `if` sobre *cómo* actuar. Conviene decirlo en clase porque es una pregunta típica del profesor.

Texto de cada diapositiva

1. Portada

- Patrón Strategy
- Integrantes / Curso / Fecha *(necesito estos datos)*

2. Patrón elegido

- Strategy (Estrategia)
- Categoría: patrón de comportamiento
- *Define una familia de algoritmos, los encapsula y los hace intercambiables.*

3. ¿Qué problema resuelve?

- Una clase con varias formas de hacer lo mismo
- Se llena de `if/else` o `switch`
- Cada variante nueva obliga a modificar la clase
- Viola el principio Abierto/Cerrado
- *(Mini código: un `if/elif` con 3 ramas)*

4. Estructura genérica

- Diagrama UML: `Context` → `Strategy` (interfaz) ← `ConcreteStrategyA / B`
- A un costado, los roles en una línea cada uno:
  - Context: usa una estrategia
  - Strategy: define el método común
  - ConcreteStrategy: implementa un algoritmo

5. Código genérico

```python
class Strategy(ABC):
    @abstractmethod
    def execute(self): ...

class ConcreteStrategyA(Strategy):
    def execute(self): ...  # algoritmo A

class ConcreteStrategyB(Strategy):
    def execute(self): ...  # algoritmo B

class Context:
    def __init__(self, strategy):
        self.strategy = strategy

    def do_work(self):
        self.strategy.execute()
```

6. Ventajas y desventajas (dos columnas)

- Ventajas: elimina condicionales / cumple Abierto/Cerrado / intercambiable en ejecución / fácil de probar
- Desventajas: más clases / el cliente debe conocer las estrategias / excesivo con solo 2 variantes fijas

7. Ejemplo 1: Métodos de pago (situación)

- Tienda online
- El cliente paga con tarjeta, PayPal o transferencia
- Cada método procesa el pago de forma distinta
- Mañana puede aparecer uno nuevo (criptomonedas)

8. Sin patrón (antes)

- Código de `Pedido` con `if/elif`
- Debajo, 2 viñetas: “Agregar un método = modificar `Pedido`” y “La clase mezcla pedido y lógica de pago”

9. Con Strategy (después)

- Código de `MetodoPago`, las 3 clases concretas y `Pedido`
- *(Si no cabe, se divide en dos diapositivas: estrategias / Pedido + uso)*

10. Patrón aplicado

- UML con nombres reales
- Tabla de equivalencia: Context = `Pedido` / Strategy = `MetodoPago` / ConcreteStrategy = `Tarjeta`, `PayPal`, `Transferencia` / Client = código que crea el `Pedido`
- Idea clave: el cliente elige la estrategia una vez

11. Ejemplo 2: Enemigo de videojuego (situación)

- Un enemigo puede ser agresivo, defensivo o sigiloso
- Su comportamiento cambia durante la partida
- Ejemplo: con poca vida, pasa de atacar a defenderse

12. Sin patrón (antes)

- Código de `Enemigo` con `if/elif`
- Viñetas: “Cada comportamiento nuevo crece la clase” y “Difícil de mantener con muchos modos”

13. Con Strategy (después)

- Código de `Comportamiento`, las 3 clases y `Enemigo`
- Salida del programa en un cuadro pequeño

14. Patrón aplicado

- UML con nombres reales

- Equivalencia: Context = `Enemigo` / Strategy = `Comportamiento` / ConcreteStrategy = `Agresivo`, `Defensivo`, `Sigiloso`
- Idea clave: la estrategia cambia en tiempo de ejecución

15. Conclusiones

- Strategy separa *qué* se hace de *cómo* se hace
- Reemplaza condicionales por clases intercambiables
- Útil cuando hay varias variantes de un algoritmo que pueden crecer

---

Para generar el PowerPoint necesito

1. **Integrantes, nombre del curso y fecha** para la portada.
2. **¿Estructura aprobada?** Si quieres agregar una diapositiva de demo por ejemplo (serían 17), dímelo. Por defecto dejo la salida del programa dentro de las diapositivas 9 y 13.

Si algo del texto o del código te parece largo o quieres cambiarlo (por ejemplo, nombres de clases), este es el momento. Con tu confirmación genero las diapositivas: fondo claro, una sola fuente, el código en cajas oscuras y los UML dibujados con formas simples.

1. Bullain Cristhian, Herbas Alan, Mendoza Fabrizio. Ingenieria de Software (A), 05/11/26
2. Aprobada.

pasame los codigos con comentarios

Preparando los códigos con comentarios.

Códigos con comentarios (Python)

Los comentarios están en español y cortos, para que quepan en las diapositivas sin saturarlas.

Código genérico (diapositiva 5)

python

```python
from abc import ABC, abstractmethod

# Strategy: define el método común a todos los algoritmos
class Strategy(ABC):
    @abstractmethod
    def execute(self): ...

# Cada ConcreteStrategy implementa un algoritmo distinto
class ConcreteStrategyA(Strategy):
    def execute(self):
        print("Algoritmo A")

class ConcreteStrategyB(Strategy):
    def execute(self):
        print("Algoritmo B")

# Context: guarda una estrategia y le delega el trabajo
class Context:
    def __init__(self, strategy):
        self.strategy = strategy

    def do_work(self):
        self.strategy.execute()   # delega, no sabe cuál es

# Client: elige la estrategia
context = Context(ConcreteStrategyA())
context.do_work()                 # Algoritmo A
context.strategy = ConcreteStrategyB()   # se cambia en ejecución
context.do_work()                 # Algoritmo B
```

Ejemplo 1: Métodos de pago

Antes, sin patrón (diapositiva 8)

python

```python
class Pedido:
    def __init__(self, monto):
        self.monto = monto

    def pagar(self, metodo):
        # Un if por cada método de pago
        if metodo == "tarjeta":
            print(f"Pagando ${self.monto} con tarjeta")
        elif metodo == "paypal":
            print(f"Pagando ${self.monto} con PayPal")
        elif metodo == "transferencia":
            print(f"Pagando ${self.monto} por transferencia")
        # Un método nuevo obliga a modificar esta clase
        else:
            raise ValueError("Método no soportado")
```

Después, con Strategy (diapositiva 9)

python

```python
from abc import ABC, abstractmethod

# Strategy: contrato que cumple todo método de pago
class MetodoPago(ABC):
    @abstractmethod
    def pagar(self, monto): ...

# ConcreteStrategy: una clase por método de pago
class Tarjeta(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} con tarjeta")

class PayPal(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} con PayPal")

class Transferencia(MetodoPago):
    def pagar(self, monto):
        print(f"Pagando ${monto} por transferencia")

# Context: el pedido no sabe cómo se paga, solo delega
class Pedido:
    def __init__(self, monto, metodo):
        self.monto = monto
        self.metodo = metodo      # estrategia recibida

    def pagar(self):
        self.metodo.pagar(self.monto)

# Client: elige el método de pago
pedido = Pedido(100, PayPal())
pedido.pagar()                    # Pagando $100 con PayPal
```

Agregar criptomonedas = crear una clase nueva, sin tocar `Pedido`.

Ejemplo 2: Enemigo de videojuego

Antes, sin patrón (diapositiva 12)

python

```python
class Enemigo:
    def __init__(self):
        self.vida = 100
        self.modo = "agresivo"

    def actuar(self):
        # Un if por cada comportamiento
        if self.modo == "agresivo":
            print("Ataca al jugador")
        elif self.modo == "defensivo":
            print("Se cubre y retrocede")
        elif self.modo == "sigiloso":
            print("Se oculta y acecha")
        # Cada modo nuevo hace crecer esta clase
```

Después, con Strategy (diapositiva 13)

python

```
from abc import ABC, abstractmethod

# Strategy: contrato que cumple todo comportamiento
class Comportamiento(ABC):
    @abstractmethod
    def actuar(self): ...

# ConcreteStrategy: una clase por comportamiento
class Agresivo(Comportamiento):
    def actuar(self):
        print("Ataca al jugador")

class Defensivo(Comportamiento):
    def actuar(self):
        print("Se cubre y retrocede")

class Sigiloso(Comportamiento):
    def actuar(self):
        print("Se oculta y acecha")

# Context: el enemigo delega su acción en la estrategia actual
class Enemigo:
    def __init__(self, comportamiento):
        self.vida = 100
        self.comportamiento = comportamiento

    def actuar(self):
        self.comportamiento.actuar()

    def recibir_dano(self, dano):
        self.vida -= dano
        if self.vida < 30:
            # Cambio de estrategia en tiempo de ejecución
            self.comportamiento = Defensivo()

# Client: crea al enemigo y juega
enemigo = Enemigo(Agresivo())
enemigo.actuar()                  # Ataca al jugador
enemigo.recibir_dano(80)          # vida baja a 20
enemigo.actuar()                  # Se cubre y retrocede
```

Salida del programa (para las diapositivas 9 y 13)

```
Pagando $100 con PayPal
```

```
Ataca al jugador
Se cubre y retrocede
```

Notas para la clase

- Las líneas `from abc import ...` pueden ocultarse en la diapositiva si no caben, pero conviene mencionarlas.
- En `Enemigo`, el `if self.vida < 30` sigue existiendo: Strategy elimina el `if` sobre *cómo actuar*, no la decisión de *cuándo cambiar*. Es una pregunta probable del profesor.
- Si quieres, puedo acortar algún bloque para que ocupe menos espacio en pantalla.

Si el código te parece bien, dime y genero el PowerPoint completo de 15 diapositivas con los datos de la portada (Bullain Cristhian, Herbas Alan, Mendoza Fabrizio, Ingeniería de Software (A), 05/11/26).

si, genera el pptx

Leyó un archivo, ejecutó un comando ▸

En pausa: tu uso gratuito vuelve a estar disponible a las 14:00. Continúa a esa hora. Continuar